3. (Problem-solving): Grammar:

S → aSa | bSb | c

Draw the **parse tree** for "abc".

Problem Statement

Given the grammar:

S → aSa | bSb | c

Draw the parse tree for the input string "abc".

Initial Analysis

First, let's analyze the grammar productions:

S → aSa: This production generates strings that are symmetric around 'a'

S → bSb: This production generates strings that are symmetric around 'b'

S → c: This is the base case production

The grammar generates palindromic strings over the alphabet {a, b, c}, where:

- Strings have odd length
- The middle character must be 'c'
- Characters appear symmetrically around the center

Attempted Derivation for "abc"

Attempt 1:

S ⇒ aSa (using S → aSa)

⇒ a(bSb)a (using S → bSb for the inner S)

⇒ ab(c)ba (using S → c for the innermost S)

⇒ abcba

Result: "abcba" ≠ "abc"

Attempt 2:

S ⇒ aSa (using S → aSa)
⇒ a(c)a (using S → c for the inner S)
⇒ aca

Result: "aca" ≠ "abc"

Attempt 3:

S ⇒ bSb (using S → bSb)
⇒ b(c)b (using S → c for the inner S)
⇒ bcb

Result: "bcb" ≠ "abc"

Fundamental Issue

The string "abc" has length 3, which is odd, so it could potentially be generated by this grammar if it were a palindrome. However, "abc" is not a palindrome since:

- First character: 'a'
- Last character: 'c'
- 'a' ≠ 'c'

For this grammar to generate "abc", it would need to have:

- First and last characters both 'a' (if derived from S → aSa)
- Or first and last characters both 'b' (if derived from S → bSb)

Since "abc" starts with 'a' and ends with 'c', it cannot be generated by this grammar

Mathematical Verification

Let's prove that "abc" ∉ L(G):

Assume for contradiction that "abc" ∈ L(G).

Case 1: If the first production used is S → aSa
Then the string must be of form: a x a, where x ∈ L(G) and |x| = 1
So "abc" must equal a x a ⇒ x = "b"
But "b" ∉ L(G) since the only terminal string S can directly produce is "c"

Case 2: If the first production used is S → bSb
Then the string must be of form: b x b, where x ∈ L(G) and |x| = 1
So "abc" must equal b x b ⇒ This is impossible since "abc" starts with 'a', not 'b'

Case 3: If the first production used is S → c
Then the string is just "c" ≠ "abc"

Therefore, "abc" cannot be generated by this grammar.

Valid Strings for This Grammar

Examples of valid strings this grammar generates:
- "c" (from S → c)
- "aca" (from S → aSa with inner S → c)
- "bcb" (from S → bSb with inner S → c)
- "aacaa" (from S → aSa with inner S → aSa with innermost S → c)
- "abcba" (from S → aSa with inner S → bSb with innermost S → c)

- "bacab" (from S → bSb with inner S → aSa with innermost S → c)

Conclusion

The string "abc" cannot be generated by the grammar S → aSa | bSb | c because:

1. The grammar generates only palindromic strings
2. "abc" is not a palindrome (first character 'a' ≠ last character 'c')
3. All strings generated by this grammar must have symmetric first and last characters

Therefore, no parse tree exists for "abc" using this grammar. The question likely contains either a typo in the string or tests understanding of grammar constraints. If the intended string was "aba", "aca", "bcb", or another palindrome ending with 'c', then a parse tree could be constructed.